

# VANGUARD AI

## Autonomous AI Security Engineer

Hackathon Build Guide | 24-Hour Execution Plan | Antigravity Master Prompt

FastAPI

React

Claude API

OWASP ZAP

Docker

01 Hour-by-Hour Timeline

03 Backend Architecture

05 AI Integration Layer

07 Demo & Judging Strategy

02 Environment Setup

04 Frontend Design System

06 Antigravity Master Prompt

Built for hackathon. Production-grade architecture. MVP scope.

# Project Overview

Vanguard AI is an AI-powered cybersecurity platform built in 24 hours for a hackathon. It combines OWASP ZAP scanning, Claude AI analysis, and a dark-luxury security dashboard to provide continuous vulnerability management — detecting, explaining, and helping fix issues before attackers can exploit them.

## Technology Stack

| Layer          | Technology                             | Purpose                          |
|----------------|----------------------------------------|----------------------------------|
| Frontend       | React + Vite + Tailwind CSS + Recharts | Dashboard, charts, AI panels     |
| Backend        | FastAPI (Python)                       | REST API, background tasks       |
| AI Engine      | Claude API (claude-sonnet-4-6)         | Explanations, fixes, chat        |
| Scanner        | OWASP ZAP REST API                     | Automated vulnerability scanning |
| Target App     | OWASP Juice Shop (Docker)              | Deliberately vulnerable demo app |
| Database       | SQLite + SQLAlchemy                    | Scan results, vuln storage       |
| Infrastructure | Docker Compose                         | All services orchestrated        |

## MVP Scope (Hackathon Deliverables)

- Scan OWASP Juice Shop using ZAP and display results on a live dashboard
- Use Claude AI to explain each vulnerability in plain English + business impact
- Generate secure code fixes (Python, JavaScript, Java) for each vulnerability
- Visualize how vulnerabilities chain into multi-step attack paths
- Provide an AI chat interface for security Q&A; with full scan context
- Generate downloadable PDF and JSON security reports
- Show compliance coverage: OWASP Top 10, PCI DSS, ISO 27001

# 24-Hour Execution Timeline

| Time Window   | Phase           | Key Deliverables                                           |
|---------------|-----------------|------------------------------------------------------------|
| 00:00 – 01:00 | SETUP           | Docker Compose, project scaffold, .env, pull images        |
| 01:00 – 03:00 | BACKEND CORE    | FastAPI app, DB models, ZAP service, seed data             |
| 03:00 – 05:30 | AI LAYER        | Claude integration, explain/fix/chat/attack-path endpoints |
| 05:30 – 08:00 | FRONTEND        | Dashboard, sidebar, vuln list, charts, score gauge         |
| 08:00 – 10:00 | AI CHAT + PATHS | Chat interface, attack path SVG, vuln detail pages         |
| 10:00 – 11:00 | REPORTS         | PDF/JSON export, compliance badges, report page            |
| 11:00 – 12:00 | POLISH + DEMO   | Loading states, demo script, rehearsal, seed refresh       |

■ Follow the timeline strictly. Start Docker before writing any code — ZAP takes 5+ minutes to download and 60 seconds to initialize. Build the dashboard before the scan page so judges always see a polished UI, even if ZAP has issues.

## Strict Build Order (Follow This)

| #  | File / Task                           | Notes                                                                    |
|----|---------------------------------------|--------------------------------------------------------------------------|
| 01 | docker-compose.yml + .env             | Run this FIRST. Juice Shop + ZAP start downloading.                      |
| 02 | backend/requirements.txt + Dockerfile | Pin all versions to avoid surprises.                                     |
| 03 | database.py + models.py               | Scan, Vulnerability, ChatMessage SQLAlchemy tables.                      |
| 04 | services/zap_service.py               | ZAP spider + active scan + alert collection.                             |
| 05 | services/ai_service.py                | Claude: explain, fix, attack path, chat, summary.                        |
| 06 | seed_data.py                          | 17 realistic Juice Shop vulns. Your demo safety net.                     |
| 07 | routers/ (all 4)                      | scans, vulnerabilities, ai, reports endpoints.                           |
| 08 | main.py                               | Wire CORS, routers, startup auto-seed, health check.                     |
| 09 | Test backend                          | curl /api/health then POST /api/scans/seed then GET /api/dashboard/stats |
| 10 | Vite + React + Tailwind setup         | Create frontend/ with all config files.                                  |
| 11 | index.css + colors                    | CSS variables, fonts, glow utilities, dark scrollbar.                    |
| 12 | api/client.js + Zustand store         | All API calls + global state.                                            |
| 13 | Layout.jsx + Sidebar.jsx              | Shell: nav, logo, status badges.                                         |
| 14 | Dashboard.jsx + all sub-components    | The hero page — spend the most time here.                                |
| 15 | VulnList.jsx + VulnDetail.jsx         | List with filters, detail with AI panels.                                |
| 16 | ScanPage.jsx                          | Progress bar, stage labels, success state.                               |
| 17 | ChatPage.jsx                          | Chat UI with suggested questions and markdown rendering.                 |
| 18 | ReportsPage.jsx                       | Download buttons, compliance progress bars.                              |

|    |                 |                                                      |
|----|-----------------|------------------------------------------------------|
| 19 | App.jsx routing | Wire all pages with React Router.                    |
| 20 | End-to-end test | seed → dashboard → vuln detail → AI → chat → report. |

# Backend Architecture

## API Endpoints Reference

| Method | Endpoint                              | Description                                 |
|--------|---------------------------------------|---------------------------------------------|
| POST   | /api/scans/start                      | Trigger ZAP scan, returns scan_id           |
| GET    | /api/scans/{id}/status                | Poll progress 0-100%, use every 3s          |
| GET    | /api/scans/{id}/results               | All vulns from a completed scan             |
| POST   | /api/scans/seed                       | Load 17 mock Juice Shop vulns (demo safety) |
| GET    | /api/vulnerabilities                  | List with ?risk=High&resolved=false filters |
| GET    | /api/vulnerabilities/{id}             | Single vuln with all AI-generated fields    |
| PATCH  | /api/vulnerabilities/{id}/resolve     | Mark as fixed, update score                 |
| POST   | /api/ai/explain/{vuln_id}             | Claude explains vuln, saves to DB           |
| POST   | /api/ai/fix/{vuln_id}?language=python | Claude generates code fix                   |
| GET    | /api/ai/attack-path/{scan_id}         | AI generates multi-step attack chain        |
| POST   | /api/ai/chat                          | Multi-turn chat with scan context           |
| GET    | /api/dashboard/stats                  | Score, counts, top vulns, compliance %      |
| GET    | /api/reports/{scan_id}/pdf            | Download PDF report                         |
| GET    | /api/reports/{scan_id}/json           | Download JSON report                        |
| GET    | /api/health                           | Health check                                |

## Database Models

| Model         | Key Fields                                                                                                                             | Notes                                   |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| Scan          | id, target_url, status, progress (0-100), security_score, critical/high/medium/low, started_at                                         | Created by ZAP, updated by script       |
| Vulnerability | scan_id, name, risk, url, param, evidence, cweid, ai_explanation, ai_fix, ai_attack_path, ai_summary, ai_score, ai_impact, is_resolved | AI fields populated via script requests |
| ChatMessage   | scan_id, role (user/assistant), content, created_at                                                                                    | Full conversation history stored        |

## ZAP Service Flow

| Stage          | Progress  | Action                                                         |
|----------------|-----------|----------------------------------------------------------------|
| Initialize     | 0%        | wait_for_zap_ready() — poll ZAP version endpoint, timeout 120s |
| Spider         | 0 → 40%   | Start ZAP spider on Juice Shop, poll until 100% spidered       |
| Active Scan    | 40 → 85%  | Start ZAP ascan, poll every 5s, update DB progress             |
| Collect Alerts | 85 → 95%  | Fetch all alerts from ZAP, save as Vulnerability records       |
| AI Analysis    | 95 → 100% | Generate attack path + executive summary via Claude            |

|          |      |                                                            |
|----------|------|------------------------------------------------------------|
| Complete | 100% | Calculate security_score, update Scan.status = 'completed' |
|----------|------|------------------------------------------------------------|

Security Score Formula

$$\text{Score} = 100 - (\text{Critical} \times 20) - (\text{High} \times 10) - (\text{Medium} \times 5) - (\text{Low} \times 2) - (\text{Info} \times 0.5)$$

Minimum score: 0. Seed data score: 31.5 / 100 (0 Critical, 6 High, 7 Medium, 4 Low)

# AI Integration Layer

## Claude System Prompt

You are Vanguard AI, an expert autonomous security engineer.  
You have deep expertise in OWASP Top 10, web application security, penetration testing, and secure software development.  
Be technical but clear. Focus on practical, actionable information.  
Always explain business impact. Be concise but complete.

## Four AI Calls to Build

### ■ explain\_vulnerability(vuln)

Returns: plain\_english, technical\_detail, attack\_scenario, business\_impact, severity\_justification. Input: name, url, param, evidence, risk, cweid, description. Result cached to vuln.ai\_explanation in DB after first call.

### ■ generate\_code\_fix(vuln, language)

Returns: VULNERABLE CODE section + SECURE CODE section with inline comments. Languages: python, javascript, java, nodejs. Cached per language in ai\_fix\_\* fields.

### ■ generate\_attack\_path(vulnerabilities)

Analyzes all scan vulns and returns JSON chain: [{step, vuln, consequence, icon}]. Also returns overall\_risk and a summary paragraph. Used by AttackPathFlow SVG component.

### ■ chat(messages, scan\_context)

Multi-turn conversation. scan\_context injected as system context: target, score, top vulns, counts. Full message history passed on every request. Returns string response.

## Seed Data — 17 Mock Vulnerabilities (Demo Safety Net)

Pre-populate the DB on startup if empty. Call POST /api/scans/seed from the Antigravity prompt or startup event. These are realistic Juice Shop findings:

| Vulnerability                    | Risk   | CWE      | URL                      |
|----------------------------------|--------|----------|--------------------------|
| SQL Injection                    | High   | CWE-89   | /rest/products/search?q= |
| Cross Site Scripting (Reflected) | High   | CWE-79   | /#/search                |
| JWT Authentication Bypass        | High   | CWE-287  | /rest/user/whoami        |
| Insecure Direct Object Reference | High   | CWE-284  | /rest/basket/1           |
| Broken Admin Access Control      | High   | CWE-285  | /api/Users               |
| Exposed Admin Interface          | High   | CWE-284  | /administration          |
| Missing Content Security Policy  | Medium | CWE-693  | /                        |
| Cleartext Transmission (HTTP)    | Medium | CWE-319  | /rest/user/login         |
| Sensitive Data in localStorage   | Medium | CWE-200  | /                        |
| Missing X-Frame-Options          | Medium | CWE-1021 | /                        |
| Outdated JavaScript Library      | Medium | CWE-937  | /main.js                 |

|                                  |        |          |                            |
|----------------------------------|--------|----------|----------------------------|
| Password Reset Token Predictable | Medium | CWE-640  | /rest/user/forgot-password |
| Missing Anti-CSRF Tokens         | Medium | CWE-352  | /api/Users                 |
| Server Information Disclosure    | Low    | CWE-200  | /                          |
| Cookie Without Secure Flag       | Low    | CWE-614  | /                          |
| Cookie Without HttpOnly Flag     | Low    | CWE-1004 | /                          |
| Verbose Error Messages           | Low    | CWE-209  | /rest/products/search?q=x' |



# Frontend Design System

## Aesthetic Direction: "Dark Threat Intelligence Terminal"

Deep navy-black backgrounds with cyan scanner accents. Monospace numbers. Subtle glow effects on active elements. Risk-color-coded everything. Feels like a real SOC terminal, not a generic SaaS dashboard.

| Token             | Hex     | Role                             |
|-------------------|---------|----------------------------------|
| --bg-primary      | #0A0E1A | Page background — deepest dark   |
| --bg-card         | #0F1628 | Card/panel background            |
| --border          | #1E2D4A | All borders and dividers         |
| --accent-cyan     | #00D4FF | Active elements, links, scanning |
| --accent-critical | #FF3366 | Critical vulnerabilities         |
| --accent-high     | #FF9500 | High severity                    |
| --accent-medium   | #FFD60A | Medium severity                  |
| --accent-low      | #00FF88 | Low severity / resolved          |
| --accent-purple   | #A855F7 | AI-powered features              |
| --text-primary    | #E8F0FF | Primary text                     |
| --text-muted      | #6B7E99 | Labels, captions                 |

### Typography

| Role      | Font                 | Usage                                   |
|-----------|----------------------|-----------------------------------------|
| Display   | Space Grotesk (Bold) | Page titles, section headers, logo      |
| Body      | Helvetica / Inter    | Body text, descriptions, labels         |
| Monospace | JetBrains Mono       | Score numbers, code blocks, data values |

## Dashboard Layout (CSS Grid)

### Row 1 Stats Bar (5 columns)

SecurityScore | Total Vulns | Critical | High | Last Scan

### Row 2 Two columns 60/40

Left: SecurityScoreGauge + ComplianceBadges | Right: RiskDonutChart

### Row 3 Full width

VulnTrendChart — line chart of last 7 scans

### Row 4 Two columns 50/50

Left: AttackPathFlow (SVG) | Right: CriticalFindings (top 5 cards)

### Row 5 Full width

AllInsightFeed — streaming Claude analysis of scan

Pages & Routes

| Route                | Component       | Primary Function                                       |
|----------------------|-----------------|--------------------------------------------------------|
| /                    | Dashboard.jsx   | Security score, charts, attack path, critical findings |
| /scan                | ScanPage.jsx    | Trigger scan, live progress bar, scan history          |
| /vulnerabilities     | VulnList.jsx    | Filterable list: risk, resolved, search                |
| /vulnerabilities/:id | VulnDetail.jsx  | ZAP data + AI explanation + code fix tabs              |
| /chat                | ChatPage.jsx    | Multi-turn AI chat with scan context                   |
| /reports             | ReportsPage.jsx | PDF/JSON download + compliance progress bars           |

# Antigravity Master Prompt

The full master prompt is in ANTIGRAVITY\_MASTER\_PROMPT.md (the companion file). Below is a structural summary of what the prompt covers across its 20 steps. Paste the complete code block from that file into Antigravity to start building.

| Step    | Target                                | Specification Detail                                                                    |
|---------|---------------------------------------|-----------------------------------------------------------------------------------------|
| Step 1  | docker-compose.yml                    | juiceshop + zap + backend + frontend services with networking                           |
| Step 2  | Backend Dockerfile + requirements.txt | FastAPI, SQLAlchemy, anthropic, python-owasp-zap-v2.4, reportlab                        |
| Step 3  | Database Models                       | Scan, Vulnerability, ChatMessage with all fields specified                              |
| Step 4  | ZAP Service                           | wait_ready, spider_target, active_scan, get_alerts, run_full_scan, calculate_score      |
| Step 5  | AI Service                            | explain_vulnerability, generate_code_fix, generate_attack_path, chat, executive_summary |
| Step 6  | Scan Router                           | start, status, results, list, seed endpoints                                            |
| Step 7  | Vuln Router                           | list with filters, detail, resolve endpoints                                            |
| Step 8  | AI Router                             | explain, fix, attack-path, chat endpoints                                               |
| Step 9  | Reports Router                        | PDF and JSON download endpoints                                                         |
| Step 10 | main.py                               | CORS, router wiring, startup auto-seed, health check                                    |
| Step 11 | Frontend Setup                        | Vite + React + Tailwind + all packages specified                                        |
| Step 12 | Design System                         | Exact hex colors, font roles, CSS variables, glow utilities                             |
| Step 13 | Layout + Sidebar                      | Shell, navigation, logo, status indicators                                              |
| Step 14 | Dashboard Page                        | All sub-components: gauge, donut, trend, attack path, findings                          |
| Step 15 | Vuln List + Detail                    | Filters, AI explain panel, code fix tabs with syntax highlighting                       |
| Step 16 | Attack Path SVG                       | SVG flowchart renderer from AI JSON chain response                                      |
| Step 17 | Chat Page                             | Multi-turn UI, suggested questions, markdown rendering, typing indicator                |
| Step 18 | Scan Page                             | 3-state UI: ready / scanning / complete with progress animation                         |
| Step 19 | Reports Page                          | Download cards, compliance progress bars                                                |
| Step 20 | State + API Client                    | Zustand store + axios client with all named functions                                   |

## How to Use the Master Prompt

1. Open Antigravity and start a new project session.
2. Paste the entire content of the code block from ANTIGRAVITY\_MASTER\_PROMPT.md.
3. Say: "Start from Step 1. Create the docker-compose.yml and .env first."
4. While Docker pulls images, say: "Now build the backend. Start from Step 2."
5. After backend test passes, say: "Now build the frontend starting from Step 11."
6. For tricky pieces (ZAP debug, PDF, SVG flowchart), switch to Claude Code (commands in the .md file).
7. Before demo: run POST /api/scans/seed and pre-generate AI explanations on top 5 vulns.

# Troubleshooting & Demo Strategy

## Common Issues + Quick Fixes

| Problem                            | Quick Fix                                                          |
|------------------------------------|--------------------------------------------------------------------|
| ZAP scan takes too long (> 20 min) | Use pre-seeded mock data. Dashboard still works perfectly.         |
| CORS errors in browser             | Add allow_origins=["*"] to FastAPI CORSMiddleware.                 |
| Claude API slow or rate limited    | Stream responses; show typing animation while waiting.             |
| ZAP returns 404 on API calls       | ZAP needs ~60s to initialize. Poll /JSON/core/view/version/ first. |
| Juice Shop unreachable             | Run: docker-compose restart juiceshop                              |
| Port conflict on 8080 or 8000      | Change host port mapping in docker-compose.yml                     |
| SQLModel relationship error        | Add Optional type annotation and check back_populates names.       |
| PDF generation crashes             | Check ReportLab version. Never use Unicode sub/superscript chars.  |
| React chart not rendering          | Wrap Recharts in a div with explicit width/height (not 100%).      |
| Zustand state not updating         | Check you're calling the action correctly, not the selector.       |

## Demo Script (Practice This)

| Time | Action      | What to Say / Show                                                                         |
|------|-------------|--------------------------------------------------------------------------------------------|
| 0:00 | Open        | Dashboard loads. Show security score 31.5, 17 vulnerabilities, colored donut chart.        |
| 0:30 | Explain     | "Our AI scanned OWASP Juice Shop — a deliberately vulnerable app. Score: 31.5 out of 100." |
| 1:00 | Drill in    | Click SQL Injection vulnerability. Show URL, parameter, evidence.                          |
| 1:30 | AI Explain  | Click "Generate Explanation". Claude explains in plain English + business impact.          |
| 2:15 | Code Fix    | Switch to Python tab. Show vulnerable code vs secure parameterized query.                  |
| 2:45 | Attack Path | Back to dashboard. Show attack chain: SQL Injection -> DB Access -> Admin Compromise.      |
| 3:15 | Chat        | Open AI Assistant. Type: "How could an attacker chain these vulnerabilities?"              |
| 3:45 | Live Scan   | Start new scan. Show animated progress bar scanning in real time.                          |
| 4:15 | Compliance  | Show OWASP / PCI DSS / ISO 27001 compliance percentages.                                   |
| 4:30 | Report      | Click "Download PDF". Show the generated security report.                                  |
| 4:45 | Vision      | "This is the foundation for an always-on AI security engineer."                            |

## Judging Talking Points

**The Problem (10s):** Traditional pen testing happens quarterly. Breaches happen daily. Organizations are blind between assessments.

**The Solution (20s):** Vanguard AI is a 24/7 autonomous security engineer — scanning continuously, explaining vulnerabilities in plain English, and generating code fixes automatically.

**The AI Magic (30s):** Claude doesn't just detect vulnerabilities — it explains the business impact, shows how an attacker would chain them together, and writes the secure code fix.

**The Stack (10s):** FastAPI + React + ZAP + Claude API. Full-stack in 24 hours. Production-grade architecture, MVP scope.

**The Vision (20s):** The roadmap is CI/CD integration, GitHub PR generation, multi-agent autonomous red-teaming, and an always-on SOC assistant. The hackathon MVP is step one.

## Definition of Done (Hackathon Checklist)

- Dashboard loads with security score and risk distribution chart
- Can trigger a ZAP scan OR show pre-seeded scan results reliably
- At least 3 vulnerabilities have Claude AI explanations rendered
- At least 1 vulnerability has a syntax-highlighted code fix (Python + JS)
- AI chat interface responds to security questions with scan context
- Attack path visualization renders (5+ step chain)
- PDF or JSON report can be downloaded
- Compliance badges visible with percentage scores
- Sidebar navigation works between all 5 pages
- No console errors in browser during demo

---

Built in 24 hours for hackathon. Production-grade architecture. MVP scope. Vanguard AI — Autonomous AI Security Engineer.